

Elixir meets TPTP

Bringing Automated Reasoning to the BEAM Ecosystem

Johannes Schuster, David Fuenmayor, Christoph Benzmüller

July 25, 2026

University of Bamberg, Chair for AI Systems Engineering

PAAR'26 @ FLoC, Lisbon

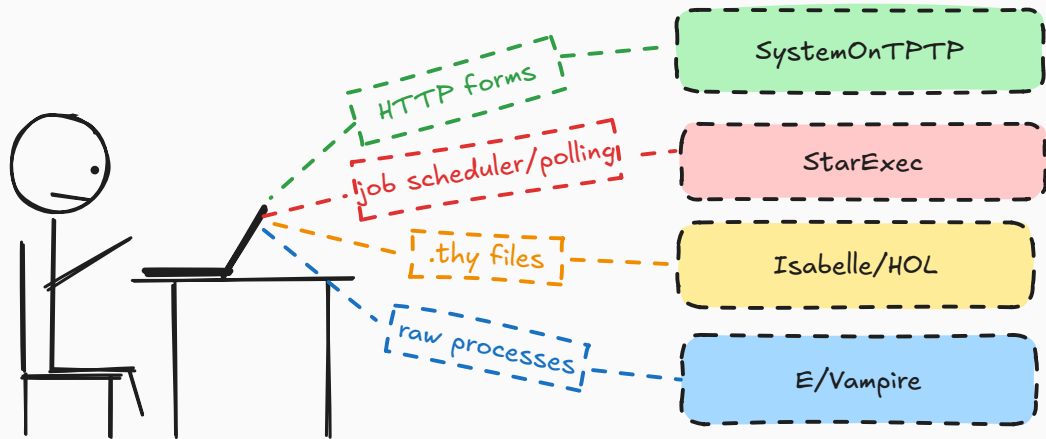


Slides



Demo Files

Integration of ATPs today



script-based, tightly coupled, no uniform way to work across backends

Standardization

Virtually all provers accept **TPTP** Output is less uniform: **SZS** gives a target, but some systems emit their own termination messages, and a host has to understand all of them.

Shared Ground Truth

Evaluating new provers requires a **separate** evaluation environment for every baseline – *SystemOnTPTP and StarExec and Isabelle and local provers.*

What we are claiming

Not a wrapper around one service. But:

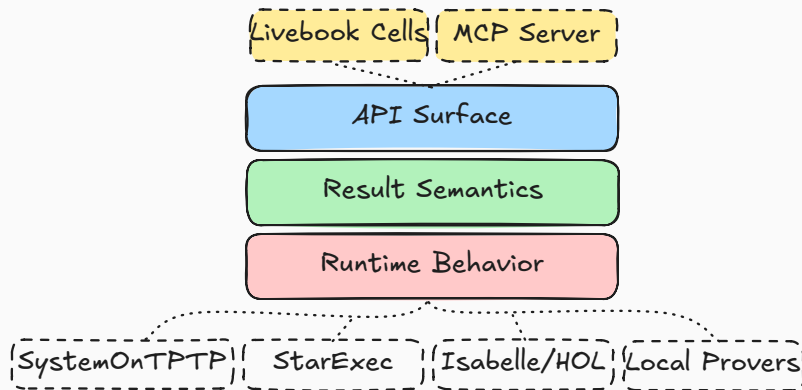
One uniform way to use four structurally different backends.

The caller states a problem and a backend, and gets back a verdict in the SZS vocabulary. Transport, polling, and output conventions stay on our side of the interface.

The caller does *not* need to supply

- a parser for that prover's output
- a cancellation mechanism
- error handling

Our contribution



Backend-specific mechanisms

	SystemOnTPTP	StarExec	Isabelle/HOL	Local
Transport	synchronous HTTP	job scheduler	TCP server	OS binary via Port
Sync mechanism	request/re-sponse	poll for completion	poll via session	wait on process
Cancellation	drop request	explicit job-deletion	session link release	kill PID
Quirk	form-encoded	multipart + redirect cookies	needs shared <code>.thy</code> on disk	Port doesn't kill child

Four columns, no two alike — and the caller should see *none* of it.



The language. Functional and gradually-typed, released 2012. Immutable data, process-oriented, pattern matching. Compiles to BEAM bytecode.



The language. Functional and gradually-typed, released 2012. Immutable data, process-oriented, pattern matching. Compiles to BEAM bytecode.



The runtime. The BEAM — Erlang's VM, built at Ericsson from 1992 for telephony. Concurrency, distribution and fault tolerance are runtime primitives, not libraries.



The language. Functional and gradually-typed, released 2012. Immutable data, process-oriented, pattern matching. Compiles to BEAM bytecode.



The runtime. The BEAM — Erlang's VM, built at Ericsson from 1992 for telephony. Concurrency, distribution and fault tolerance are runtime primitives, not libraries.



The ecosystem. Phoenix (web), Nx (numerical computing), Livebook (notebooks). #3 most admired language, Phoenix: #1 web framework (Stack Overflow 2025).

A unified result type

```
1 problem = "thf(funExt, conjecture,  
2     ![F:$i>$i, G:$i>$i]: ( ( ![X:$i]: ( ( F @ X) = ( G @ X) ) ) => ( F = G )  
3 )."  
4  
5 AtpClient.SystemOnTptp.query_system(problem, "Leo-III---1.8.0")  
6 # => {:ok, :theorem}  
7  
8 AtpClient.SystemOnTptp.query_selected_systems(problem,  
9     ["Zipperpin---2.1", "E---3.3"])  
10 # => {:ok, [{"Zipperpin---2.1", {:ok, :theorem}}],  
11 #     {"E---3.3", {:ok, :theorem}}]}}  
12  
13 AtpClient.Isabelle.query(problem, proof_method: "by auto")  
14 # => {:ok, :theorem} # same problem, same verdict, different backends
```

Elixir in ten seconds: `:theorem` is an *atom* (a symbolic constant); `{:ok, :theorem}` is a *tuple*; callers *pattern-match* on these shapes.

One caveat before you trust the verdict

Our atoms are SZS statuses lowercased — `:theorem`, `:counter_satisfiable`, `:timeout`, `:gave_up`, and so on.

`:theorem` is the tool's *claim* of success — nothing more. It is kernel-verified only when the proof method itself runs through the kernel (e.g. `by auto`); sledgehammer verdicts are not re-checked.

How to proceed with non-TPTP backends and provers that don't emit SZS tags?

Classification layers:

1. Explicit `% SZS status <S>` line
2. Permissive SZS fallback (any CamelCase status name)
3. Prover-specific patterns — 24 patterns / 7 provers, following sledgehammer's known-failures tables (Alt-Ergo, E, iProver, LEO-II, SPASS, Vampire, Waldmeister)
4. Otherwise: `{:error, {:unrecognized_output, raw}}` (no guessing).
`raw: true` bypasses classification entirely, on every backend.

Isabelle's tool output can also be directly mapped to SZS.

Case study: why we under-claim

SPASS beiseite: Completion found: the given clause set was saturated under the strategy applied.

Our post-hoc parser can't recover

- the prover's *mode* (SAT or refutational proof).
- the (possibly incomplete) strategy producing the model.

Both facts are known *inside* the prover; neither survives the trip to stdout. So we record **:gave_up** — agreeing with sledgehammer.

Benefits of the runtime

ATP orchestration needs

run a portfolio of provers at once

The BEAM ships

↔ lightweight preemptive processes

Benefits of the runtime

ATP orchestration needs

run a portfolio of provers at once
one backend dying must not kill the host

The BEAM ships

↔ lightweight preemptive processes
↔ supervision trees, “let it crash”

Benefits of the runtime

ATP orchestration needs

run a portfolio of provers at once
one backend dying must not kill the host
“stop everything I started”

The BEAM ships

↔ lightweight preemptive processes
↔ supervision trees, “let it crash”
↔ process links — death *is* cancellation

Benefits of the runtime

ATP orchestration needs

run a portfolio of provers at once
one backend dying must not kill the host
“stop everything I started”
branch on a dozen result shapes, readably

The BEAM ships

↔ lightweight preemptive processes
↔ supervision trees, “let it crash”
↔ process links — death is cancellation
↔ pattern matching as control flow

Benefits of the runtime

ATP orchestration needs		The BEAM ships
run a portfolio of provers at once	↔	lightweight preemptive processes
one backend dying must not kill the host	↔	supervision trees, “let it crash”
“stop everything I started”	↔	process links — death is cancellation
branch on a dozen result shapes, readably	↔	pattern matching as control flow
poke at a prover interactively	↔	Livebook

The Elixir ecosystem already handles most of our needs.

Example: portfolio solving as a single combinator

```
1 problem = "fof(ident, conjecture, ![X, Y]: ((X = Y) => (f(X) = f(Y)))))."
2 AtpClient.SystemOnTptp.list_provers()
3 |> Task.async_stream(
4   fn sys -> {sys, AtpClient.SystemOnTptp.query_system(problem, sys)} end,
5   ordered: false,
6   timeout: 30_000, on_timeout: :kill_task)
7 |> Enum.find(&match?({:ok, {_sys, {:ok, :theorem}}}, &1))
8 # => {:ok, {"Darwin---1.4.5", {:ok, :theorem}}} (Non-deterministic)
```

- **Task.async_stream**: each query is its own preemptively-scheduled process
- **on_timeout: :kill_task**: all unfinished queries are killed at its deadline (the timeout per task); the others are unaffected
- **ordered: false + Enum.find**: the first success wins as it arrives; halting the stream shuts the remaining tasks down

Two consumers, no backend code

KinoAtpClient — Livebook Smart Cell

A TPTP IDE in a notebook: live linting, hover docs, one-click solve, and a backend dropdown — all four, same editor.

AtpMcp — MCP server

All four backends exposed to LLM agents as a single **query_backend** tool.

Neither needs to know any specifics of *StarExec* or *Isabelle*. Both talk only to the abstract **AtpClient.Backend** behavior — one **query/2** callback that every backend implements.

Related work — and what we adapted

	What it does well	Where we sit
sledgehammer	prover-specific output tables	we borrowed them — and generalized the strategy beyond Isabelle to arbitrary hosts
Why3	powerful goal transformations	we don't touch the goal; logistics only , different host language
OnlineATPs / atp / tptp (Haskell)	CLI and parse-level building blocks	a real TPTP parser like tptp would strengthen our linter's structural tier
TPTP Editor (VS Code)	closest editor tool	two-tier linting (structural + TPTP4X); lives in a notebook sharing runtime state with neighbouring cells

Recap: uniform result type across four backends · fault-tolerant concurrency · thin notebook and agent surfaces

Future work:

- **more backends** (incl. non-TPTP bridges) and per-call metadata (runtime, proof statistics, proof object/model etc.)
- a BEAM-native **agent framework** where human and LLM agents collaborate on a shared, SZS-grounded proof state
- a Livebook-based next-generation **ITP system**: TPTP-based or with a new specification language

Our releases

AtpClient — *use your favorite theorem prover*

Hex: `atp-client.hexdocs.pm`

GitHub: `github.com/jcschuster/AtpClient`



KinoAtpClient — *notebook front-end with syntax highlighting*

Hex: `kino-atp-client.hexdocs.pm`

GitHub: `github.com/jcschuster/KinoAtpClient`



AtpMcp — *ATP tools for LLM agents*

Hex: `atp-mcp.hexdocs.pm`

GitHub: `github.com/jcschuster/AtpMcp`

